연구논문/작품 중간보고서

2026 학년도 제 2 학기

제목	GPU 프로파일링·시뮬레이션 기반 LLM 추론 attention 커널 동적 선택 시스템	○ 논문( ) 작품(✓) ※해당란 체크
GitHub URL	https://github.com/sunjae0224/kernelscope	
평가등급	**지도교수 수정보완 사항**	**팀원 명단**
A,B,F 중 택 1 (지도교수가 부여)	○ ○ ○	이 선 재 (인) (학번: 2021313805)

2026 년 10 월 8 일

지도교수 : 민 동 문 서명_____

■ 요약

본 작품은 길이가 서로 다른 요청이 한 디코드 배치에 섞이는 LLM 서빙 환경에서 attention 커널의 실행 설정(KV 분할 수와 커널 변형)을 매 디코드 단계마다 동적으로 선택하는 시스템이다. FlashAttention-2 의 디코드 경로는 배치 크기·헤드 수·최대 KV 길이만으로 분할 수를 정하고, FlashInfer 는 헤드 그룹 크기만으로 tensor-core 변형을 기본으로 선택한다. 두 라이브러리의 기본 설정 모두 요청별 길이 분포를 반영하지 않으므로, 소수의 긴 요청과 다수의 짧은 요청이 섞이면 일부 스레드 블록에 작업이 집중되는 부하 불균형 (load imbalance)이 발생한다.

RTX 4090 에서 커널 7,443 개 조건을 측정한 결과, 균일 길이 배치에서는 기본 휴리스틱이 최적에 가까웠지만 (최적 대비 손실 중앙값 연속 KV 0.72%, 페이지 KV 1.2%) 가변 길이 배치에서는 최적 분할 수보다 중앙값 1.44 배, 최악 3.98 배(페이지 KV 기준, 연속 KV 는 12.53 배) 느린 커널이 선택되었다. 긴 요청이 64K 토큰이면 손실은 중앙값 2.84 배, 최악 5.18 배로 커졌다. FlashInfer 의 기본 변형도 32K 격자에서 최악 2.49 배, 64K 격자에서 최악 2.99 배 느렸다. 공개 요청 트레이스(Azure LLM Inference 2023, BurstGPT)를 연속 배치로 재생하면 이 손실은 긴 프롬프트의 비중과 배치 크기 (26 이상)에 따라 전체 디코드 단계의 0.3~14%에서 나타나는 조건부 문제였다.

실측 조회 테이블 정책은 Qwen3-4B 의 32K×1+512×31 조건에서 토큰 간 지연(TPOT)을 61.06 ms 에서 33.80 ms(1.81 배)로, 64K 조건에서는 32K 측정값을 외삽하여 96.93 ms 에서 40.06 ms(2.42 배)로 줄였다. 실측값과 성능 모델을 결합한 하이브리드 정책은 이 조건들에서 테이블과 같은 분할 수를 골라 같은 수준 (1.79 배)이었다. 같은 엔진에 FlashInfer 를 통합하여 비교하면, 커널을 교체한 CUDA-core 변형(1.83 배)과 기존 커널의 분할 수만 외부에서 선택한 결과(1.81 배)가 같은 수준이었고 FlashInfer 기본 변형은 1.26 배에 그쳤다. 즉 커널과 엔진을 바꾸지 않고 실행 설정만 선택해도 커널 교체 이득의 대부분을 얻는다.

출력 동일성은 토큰 일치율 대신 탐욕적 디코딩 경로가 갈라지는 지점의 수와 teacher forcing 으로 측정한 로짓 여유(bf16 ULP 단위)로 보고한다. 32K 조건의 테이블·하이브리드 정책은 분기 0 건이었고, 2 ULP 를 넘는 분기는 고정 분할 대조군과 64K 조건의 두 사건뿐이었다. 두 사건에 대해 같은 KV 캐시 상태에서 층별 attention 출력을 fp32 참조와 비교한 커널 수준 검사(네 차례)에서 커널 간 차이는 모두 bf16 1 ULP 이내였다. 즉 검사한 사건에서 커널 간 차이는 반올림 수준이며, 생성 경로의 변화는 그 차이가 36 층과 수십 단계에 걸쳐 증폭된 결과이다. GPU 없이 원본 기록에서 문서의 수치 531 개를 재계산하는 검증이 모두 통과하였다. 남은 기간에는 vLLM 에서 관찰된 가변 길이 손실의 원인 분리, Accel-Sim L2 설정 재검증, 최종 시연을 진행한다.

주요어: LLM 추론, 연속 배치, FlashAttention, FlashInfer, KV 분할, 부하 불균형, GPU 성능 분석

■ 1. 서론

가) 제안배경 및 필요성

(1) LLM 서빙과 디코드 attention

Transformer[1] 구조에 기반한 대규모 언어 모델이 검색, 대화, 코드 작성 등 다양한 서비스에 활용되면서, 모델을 학습시키는 비용뿐 아니라 이미 학습된 모델로 답을 생성하는 추론(inference) 비용이 서비스 운영의 핵심 비용이 되었다. LLM 의 추론은 입력 문장 전체를 한 번에 처리하는 프리필(prefill) 단계와, 이후 토큰을 하나씩 생성하는 디코드(decode) 단계로 나뉜다. 디코드 단계는 생성할 토큰 수만큼 반복되므로 전체 생성 시간의 대부분을 차지하며, 사용자는 이를 토큰 간 지연(TPOT, time per output token)으로 체감한다.

GPU 한 장이 처리할 수 있는 연산량에 비해 요청 하나의 디코드 연산은 매우 작기 때문에, 서빙 시스템은 여러 요청을 하나의 배치로 묶어 처리한다. Orca[2]는 요청 단위가 아니라 디코드 반복(iteration) 단위로 배치를 재구성하는 스케줄링을 제안하였고, vLLM[3]은 요청마다 길이가 다른 KV 캐시를 고정 크기 페이지 단위로 관리하는 PagedAttention 을 제안하였다. 이러한 연속 배치(continuous batching) 방식에서는 서로 다른 시점에 도착하여 서로 다른 길이의 문맥을 가진 요청들이 같은 배치 안에서 함께 처리된다. 실제 서비스에서는 긴 문서를 요약하는 요청과 짧은 대화 요청이 섞이는 것이 일반적이므로, 한 배치 안의 KV 캐시 길이는 수십 배 이상 차이가 날 수 있다.

디코드 단계의 attention 은 새로 생성할 토큰 하나의 질의(query) 벡터가 지금까지 저장된 모든 키(key)와 값(value)을 읽어 가중합을 구하는 연산이다. 한 번 읽은 데이터에 대해 수행하는 연산이 적기 때문에 메모리 대역폭에 의해 성능이 제한되며, 병렬로 처리할 수 있는 작업의 양은 질의 길이가 아니라 배치 크기와 헤드 수에 의해 결정된다. 배치가 작거나 헤드 수가 적으면 GPU 의 스트리밍 멀티프로세서(SM)를 모두 채우지 못한다. Flash-Decoding[4]은 이 문제를 해결하기 위해 각 요청의 KV 캐시를 여러 조각으로 나누어 서로 다른 스레드 블록이 병렬로 처리한 뒤 결과를 합치는 KV 분할(split-KV) 방식을 제안하였고, 이 방식은 FlashAttention-2[5] 라이브러리의 디코드 경로에 포함되어 널리 사용되고 있다.

본 작품은 이 디코드 attention 을 대상으로 한다. 이미 구성된 배치와 모델 가중치를 유지한 채 커널의 실행 설정만 바꾸므로, 배치 구성이나 메모리 관리 정책과 분리하여 효과를 측정할 수 있다. 관심 지표는 커널 실행 시간뿐 아니라 실제 생성 과정에서 사용자가 체감하는 TPOT 이다.

(2) 요청 길이 차이와 라이브러리 기본 설정의 문제

KV 캐시를 몇 조각으로 나눌지(분할 수, num_splits)는 라이브러리가 휴리스틱으로 정한다. FlashAttention-2 2.8.x 의 디코드 경로(flash_attn_with_kvcache)는 num_splits=0 일 때 배치 크기×KV 헤드 수×질의 블록 수가 SM 당 2 개 CTA 를 가정한 실행 슬롯 수의 80%를 넘으면 요청별 KV 길이를 보지 않고 분할 없이 실행한다. 즉 배치 안의 모든 요청이 비슷한 작업량을 가진다고 가정한다. FlashInfer[14]의 디코드 래퍼는 grouped-query attention(GQA)의 그룹 크기가 4 이상이면 tensor-core 변형을 권장하며 이 역시 길이 분포를 보지 않는다. 두 기본 설정 모두 길이가 섞인 배치에서는 소수의 긴 요청을 담당한 스레드 블록에 작업이 집중되고, 짧은 요청을 담당한 블록은 조기에 종료되어 나머지 SM 이 유휴 상태가 되는 부하 불균형을

일으킨다. 이러한 비효율은 평균 지연 시간과 같은 시스템 수준의 지표만으로는 원인을 파악하기 어려우며, 커널 수준의 측정과 GPU 실행 구조에 대한 분석이 함께 필요하다.

또한 한 종류의 GPU 에서 얻은 측정만으로는 SM 수나 메모리 대역폭이 다른 하드웨어에서 같은 현상이 어떻게 나타날지 답할 수 없다. 이를 위해서는 GPGPU-Sim[6], Accel-Sim[7]과 같은 사이클 수준 GPU 시뮬레이터나 실측으로 보정한 성능 모델을 활용하여 하드웨어 조건을 바꾸어 보는 민감도 분석 (sensitivity analysis)이 필요하다.

(3) 문제의 발생 조건

이 손실은 모든 배치에서 생기는 것이 아니다. 공개된 실제 요청 트레이스(Azure LLM Inference 2023[18], BurstGPT[19])를 연속 배치 스케줄러로 재생하여 매 디코드 단계의 길이 분포를 채점하면, 채팅 트레이스를 GPU 한 장이 받는 부하에서는 전체 단계의 13.5%가 attention 시간을 1.25 배 이상 잃지만, 프롬프트가 짧은 트레이스에서는 0.3%에 그친다. 손실이 나타나는 조건은 배치 크기가 분할 없음 임계(헤드 구성 32/8 에서 배치 26 이상)를 넘고 배치 안에 긴 프롬프트가 포함되는 경우이며, 긴 요청의 길이가 8K 에서 64K 로 늘수록 손실은 단조 증가한다. 따라서 본 작품은 이 문제를 드문 극단이 아니라 긴 문맥 비중과 배치 크기에 달린 조건부 문제로 정의하고, 그 조건과 크기를 함께 보고한다. 이상을 정리하면 본 작품의 필요성은 다음과 같다.

- 실제 서빙 조건인 가변 길이 배치에서 라이브러리 기본 설정이 얼마나 손해를 보는지, 그리고 그 손실이 실제 트래픽에서 얼마나 자주 나타나는지 정량적으로 측정할 필요가 있다.
- 그 원인을 스레드 블록 구성과 SM 점유율, 메모리 대역폭 등 GPU 실행 구조의 관점에서 설명하고, 디코드 단계 전체에서 attention 이 차지하는 비중으로 개선 상한을 확인할 필요가 있다.
- 커널 수준의 개선이 실제 사용자가 체감하는 TPOT 으로 이어지는지 전체 모델의 생성 과정에서 검증하고, 커널을 교체하는 기존 해법(FlashInfer)과의 격차를 같은 조건에서 측정할 필요가 있다.
- 설정 변경이 생성 결과를 바꾸는지, 바뀐다면 그 원인이 커널 구현 오류인지 부동소수점 반올림의 증폭인지 구분하여 검증할 필요가 있다.

따라서 단순히 가장 빠른 커널을 찾는 것만으로는 충분하지 않다. 동일한 입력에서 후보를 비교하고, 선택에 드는 CPU 시간과 전체 디코드 시간을 함께 기록하며, 생성 결과가 유지되는 조건과 달라지는 조건을 구분해야 한다. 이러한 기준을 측정 도구와 실제 모델 실행 엔진에 공통으로 적용하였다.

나) 연구논문/작품의 목표

(1) 최종 목표와 세부 목표

본 작품의 최종 목표는 요청 길이 분포에 따라 attention 커널의 실행 설정을 매 디코드 단계마다 동적으로 선택하는 시스템을 구현하고, 그 효과와 한계를 실제 LLM 의 생성 과정에서 검증하는 것이다. 새로운 attention 수식이나 새로운 커널을 만드는 것이 아니라, 수학적으로 같은 결과를 내는 기존 커널의 실행 설정(FlashAttention-2 의 분할 수, 선택적으로 FlashInfer 의 커널 변형)만을 바꾼다는 점이 특징이다. 설정이 달라도 연산은 같지만 부동소수점 합산 순서가 달라 출력이 반올림 수준에서 달라질 수 있으므로, 성능 개선은 생성 결과의 동일성 검증과 함께 보고한다. 세부 목표와 중간 시점의 진행 상황은 표 1 과 같다.

표 1. 세부 목표와 중간 시점 진행 상황

구분	목표	중간 시점 결과(2026-10-08)
커널 측정	동일 입력으로 분할 수·KV 형식·라이브러리 변형을 비교	7,443 개 조건(FlashAttention-2 9 종, FlashInfer 2 종, 8K~64K), 오류 0 건
문제의 범위	실제 트래픽에서의 발생 빈도와 문맥 길이에 따른 추세	공개 트레이스 3 종 재생(20 개 구성), 64K 격자 추세, 두 라이브러리 기본 설정 손실
원인 분석	CTA 작업량·연산 클래스 분해·시뮬레이션으로 원인 설명	부하 불균형 확인, decode step 8 개 연산 클래스 분해, Accel-Sim 43 건·GPGPU-Sim 축소 커널 재현
선택 정책	휴리스틱·고정값·테이블·모델·하이브리드·라이브러리 교체 정책 구현	7 종 정책 구현, 하이브리드 정책 leave-one-out 최악 손실 cold 집합 10.9% 이내
실제 모델 검증	TPOT 과 생성 토큰 동일성을 함께 평가	Qwen3-4B 32K 1.81 배·64K 2.42 배, 하이브리드 1.79 배, FlashInfer 통합 비교, vLLM 재현
출력 검증	불일치의 원인을 커널 수준에서 판정	분기 지점·ULP 여유 분류 도입, 2 ULP 초과 사건 2 건의 커널 수준 검사(4 회) 모두 1 ULP 이내
재현과 시각화	GPU 없이 결과 재계산 및 탐색	531 개 검증 통과, 대시보드 시연 페이지 + 분석 탭 5 개

(2) 비교 범위와 평가 기준

후보는 동일한 attention 계산을 수행하는 FlashAttention-2 의 실행 변형 9 종(분할 수 1, 기본 휴리스틱, 고정 분할 수 2~128)과 FlashInfer 의 두 변형(tensor-core, CUDA-core)이다. 연속 KV 캐시와 페이지 단위 KV 캐시는 별도로 비교하며, 캐시를 비운 cold 상태와 재사용하는 warm 상태도 구분한다. 커널 선택의 손실은 선택한 실행 시간을 같은 조건의 최적 후보 시간으로 나눈 값(최적 대비 배율)으로 정의하며, 이는 자동 튜닝 문헌의 regret 에 해당한다.

실제 모델에서는 같은 가중치·입력·요청 순서와 도착 시점을 사용하고 정책만 교체한다. 생성 토큰이 기본 휴리스틱과 같은 경우를 출력 동일성이 확인된 개선으로 보고한다. 토큰이 달라진 경우에는 탐욕적 디코딩 경로가 처음 갈라진 지점(분기 지점)의 수를 세고, 입력을 고정한 teacher forcing 진단으로 기준 정책의 1 위 로짓과 후보 정책이 고른 토큰의 로짓 차이를 bf16 ULP(unit in the last place) 단위로 분류한다. 2 ULP 를 넘는 차이는 같은 KV 캐시 상태에서 층별 attention 출력을 fp32 참조와 비교하는 커널 수준 수치 검사로 구현 오류 여부를 판정한다.

다) 연구논문/작품 전체 overview

(1) 시스템 구성

시스템은 커널 측정, 원인 분석, 설정 선택, 실제 모델 실행, 결과 검증, 재현과 시각화의 여섯 단계로 구성된다. 입력은 한 배치에 포함된 요청들의 KV 길이 분포이며, 선택 정책은 매 디코드 단계에서 이 정보를 받아 분할 수(그리고 라이브러리 교체 정책에서는 커널 변형)를 반환한다. 그림 1 은 각 단계의 관계를 보여 준다.

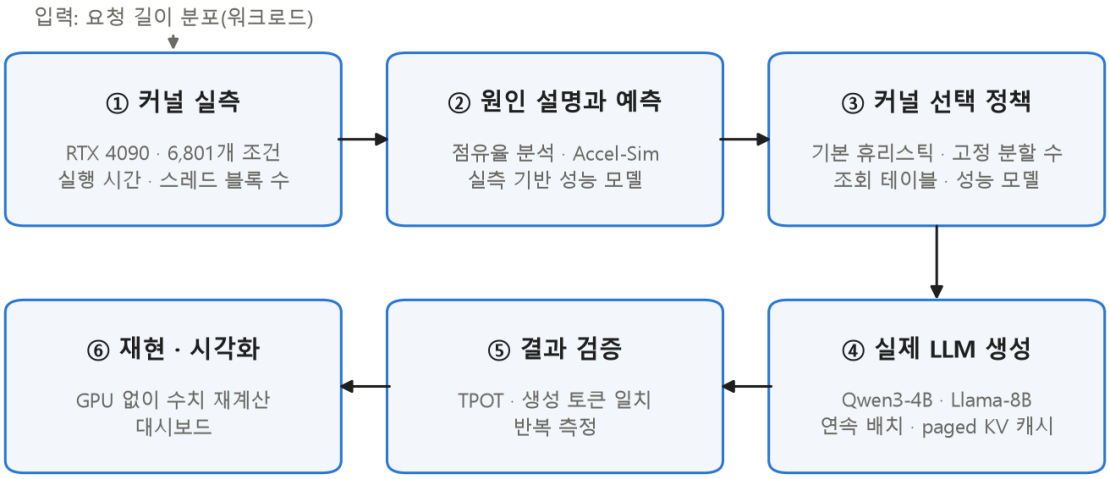


그림 1. 요청 길이 분포에 따른 attention 커널 선택과 검증 과정

커널 실측은 RTX 4090 에서 후보별 실행 시간과 CTA 구성을 기록한다. 분석 단계는 CTA 작업량, 디코드 단계의 연산 클래스별 시간 분해, Accel-Sim 과 실측 기반 성능 모델을 이용하여 병목의 원인을 설명하고, 공개 트레이스 재생으로 문제의 발생 빈도를 추정한다. 선택 정책은 실측값 또는 예측값을 바탕으로 설정을 고르며, 실제 모델 엔진은 Qwen3-4B 와 Llama 계열 8B 를 연속 배치 및 페이지 단위 KV 캐시로 실행한다. 엔진에는 FlashAttention-2 와 FlashInfer 두 백엔드가 통합되어 같은 규약으로 비교된다.

결과 검증에서는 TPOT 과 생성 토큰 동일성을 반복 측정하고, 불일치는 분기 지점과 ULP 여유로 분류한다. 원본 기록은 분석 및 시각화 단계로 전달되므로 GPU 를 확보하지 못한 환경에서도 주요 수치를 다시 계산하고 대시보드로 확인할 수 있다. 측정과 재현 단계의 분리는 실험 장비와 보고서 작성 환경이 달라도 같은 근거를 확인하기 위한 설계이다.

(2) 중간 시점의 주요 결과와 보고서 구성

첫째, 균일 길이 배치에서는 기본 휴리스틱의 최적 대비 손실 중앙값이 0.72%로 작았지만 가변 길이 배치에서는 페이지 KV 기준 중앙값 1.44 배, 최악 3.98 배로 커졌고, 긴 요청이 64K 이면 중앙값 2.84 배, 최악 5.18 배에 이르렀다. FlashInfer 의 기본 변형도 같은 32K 격자에서 최악 2.49 배, 64K 격자에서 최악 2.99 배 느렸다. CTA 별 작업량 분석은 긴 요청을 담당한 소수의 블록에 작업이 집중되는 것이 원인임을 보여 주었다.

둘째, 공개 트레이스 재생에서 손실이 1.25 배를 넘는 디코드 단계의 비율은 Azure 채팅 트레이스 13.5%, 코드 트레이스 14.1%, BurstGPT 0.3%였다. 문제는 배치 크기 26 이상과 긴 프롬프트가 함께 있을 때 나타나며, 프롬프트 길이를 4 배로 늘리고 KV 예산을 436K 토큰으로 가정한 가상 조건에서는 39.5%까지 커졌다.

셋째, 조회 테이블 정책은 Qwen3-4B 의 가변 길이 조건에서 TPOT 을 1.81 배(32K), 2.42 배(64K, 32K 측정값 외삽), 실행 도중 요청이 추가되는 조건에서 1.17 배 개선하였고, 하이브리드 정책은 이 조건들에서 테이블과 같은 분할 수를 골라 1.79 배로 같은 수준이었다. 디코드 단계의 연산 클래스 분해에서 attention 비중은 70.9%에서 37.5%로 낮아졌고, 균일 길이에서는 attention 비중이 19.2%에 불과하여 선택의 여지가 작았다.

넷째, FlashInfer 를 같은 엔진에 통합하여 비교한 결과, CUDA-core 변형으로 커널을 교체한 경우(1.83 배)와 기존 커널의 분할 수만 선택한 경우(1.81 배)가 같은 수준이었고, 라이브러리까지 함께 고르는 조회 테이블의 추가 이득은 약 1%였다. 매 단계 수백 μs 가 드는 FlashInfer 의 plan 단계 비용이 그 한계를 정한다.

다섯째, 출력 불일치는 분기 지점 수와 ULP 여유로 분류하였다. 32K 조건의 테이블·하이브리드 정책은 분기 0 건이었고, 분류에서 2 ULP 를 넘은 두 사건(고정 분할 대조군, 64K)은 네 차례의 커널 수준 수치 검사에서 층별 출력이 1 ULP 이내로 일치하여 구현 오류가 아님을 확인하였다. GPU 없이 원본 기록을 재계산하는 검증은 531 개 항목이 모두 통과하였다.

2 장에서는 LLM 서빙, attention 커널 최적화, GPU 성능 분석에 관한 연구와 원안과의 관계를 정리한다. 3 장에서는 이론적 배경, 시스템 구성과 구현, 선택 정책과 출력 검증 절차를 설명한다. 4 장에서는 커널 측정, 트레이스 재생, 실제 모델 생성, FlashInfer 비교, 출력 동일성, 성능 모델과 시뮬레이터 검증, 한계와 보완 계획을 분석한다. 5 장은 결론과 연구 과정에서 얻은 경험을 정리한다.

■ 2. 관련연구

가) LLM 추론과 서빙 시스템

Transformer[1]는 순환 구조 없이 attention 만으로 시퀀스를 처리하는 구조로, 현재 대부분의 대규모 언어 모델의 기반이 되었다. 자기회귀 방식으로 토큰을 생성할 때에는 이전 토큰들의 키와 값을 KV 캐시에 저장해 두고 재사용한다. 본 작품의 실험에 사용한 Qwen3[8]는 0.6B 부터 235B 규모까지의 dense 및 MoE 모델로 구성된 공개 모델 계열로, 여러 질의 헤드가 하나의 키·값 헤드를 공유하는 grouped-query attention(GQA) 구조를 사용한다.

Orca[2]는 요청 전체가 끝날 때까지 기다리지 않고 디코드 반복 단위로 배치를 다시 구성하는 반복 단위 스케줄링(iteration-level scheduling)을 제안하여, 길이가 서로 다른 생성 요청을 효율적으로 함께 처리할 수 있게 하였다. vLLM[3]은 운영체제의 가상 메모리처럼 KV 캐시를 고정 크기의 블록 단위로 관리하는 PagedAttention 을 제안하여 가변 길이 KV 캐시로 인한 메모리 낭비를 크게 줄였다. Sarathi-Serve[9]는 긴 프리필 요청을 여러 조각으로 나누어 진행 중인 디코드와 함께 처리하는 스케줄링으로 처리량과 지연 시간 사이의 균형을 개선하였다. 이들 연구가 확립한 연속 배치와 페이지 단위 KV 캐시 구조가 곧 본 작품이 다루는 가변 길이 배치가 발생하는 환경이다. vLLM 의 FlashAttention 백엔드는 CUDA Graph 모드에서만 분할 수의 상한을 고정하고 그 밖에는 라이브러리 휴리스틱을 그대로 사용하므로[22], 운영 엔진도 같은 기본 설정을 물려받는다.

Splitwise[18]는 Azure 의 LLM 추론 서비스에서 수집한 채팅·코드 요청 트레이스를 공개하였고, BurstGPT[19]는 ChatGPT·GPT-4 서비스의 요청 도착과 길이 분포를 수집하였다. 두 트레이스는 프롬프트 길이 분포가 크게 달라(Azure 채팅 중앙값 1,020 토큰, BurstGPT 중앙값 502 토큰), 본 작품이 다루는 가변 길이 배치가 실제 트래픽에서 얼마나 자주 생기는지를 추정하는 데 사용하였다.

국내 연구에서 김수진·유헌창[10]은 질의 간 의미적 유사도로 요청을 군집화하고 KV 캐시를 미리 구축·재사용하는 시스템을 제안하였다. 최우형 외[11]는 GPU 메모리 용량의 한계를 보완하기 위해 GPU·호스트 메모리와 저장장치를 계층적으로 활용하는 사례를 정리하였다. 전자는 요청 사이의 재사용 가능성을, 후자는 KV 데이터가 저장되는 위치를 중심으로 서빙 비용을 줄이는 접근이다.

이들 연구는 본 작품의 실행 환경과 문제 범위를 구분하는 기준이 된다. 본 작품은 캐시에 보존할 데이터나 배치에 포함할 요청을 바꾸지 않고, 이미 구성된 배치의 attention 을 단일 GPU 에서 실행할 때 실행 설정을 선택한다. 따라서 요청 스케줄링이나 KV 메모리 관리의 효과와 커널 실행 설정의 효과를 별도로 측정한다.

나) attention 커널 최적화

FlashAttention[12]은 GPU 의 고대역폭 메모리(HBM)와 온칩 SRAM 사이의 데이터 이동을 최소화하는 타일링 기법으로, 근사 없이 정확한 attention 을 빠르고 메모리 효율적으로 계산하였다. FlashAttention-2[5]는 스레드 블록과 워프 사이의 작업 분할 방식을 다시 설계하여 GPU 점유율을 높이고 처리량을 개선하였다. Flash-Decoding[4]은 디코드 단계에서 질의 길이가 1 이기 때문에 배치가 작으면 SM 을 충분히 활용하지

못하는 문제를 지적하고, KV 캐시를 여러 조각으로 나누어 병렬로 계산한 뒤 log-sum-exp 방식으로 부분 결과를 합치는 방법을 제안하였다. 본 작품이 선택하는 대상인 분할 수가 바로 이 방법의 핵심 설정값이다.

FlashDecoding++[13]는 부분 softmax 의 동기화 비용, 작은 행렬 곱의 낮은 활용률, 고정 실행 방식의 비효율을 unified max value, flat GEMM 최적화, 입력·하드웨어별 휴리스틱 선택으로 개선하였다. 입력 형태에 따라 실행 방식을 선택한다는 점은 본 작품과 공통적이다.

FlashInfer[14]는 다양한 KV 형식을 블록 희소 행렬 형식으로 통일하고, CPU 에서 수행하는 plan 단계에서 요청별 KV 를 일정 크기의 조각으로 나누어 CTA 별 비용이 균형을 이루도록 배정하는 부하 분산 스케줄링과 JIT 컴파일 기반 attention 템플릿을 제공한다. LeanAttention[20]은 디코드 attention 을 stream-K 방식의 reduction 으로 재구성하여 요청 길이가 달라도 작업을 SM 수에 맞게 균등하게 나눈다. 두 연구는 가변 길이 배치의 부하 불균형을 커널 내부 스케줄링으로 없애는 커널 수준 해법이며, 본 작품이 다루는 현상의 상한에 해당한다. 반면 본 작품은 기존 FlashAttention-2 커널을 그대로 두고 분할 수 인자를 외부에서 선택하며, 그 이득이 커널 교체의 이득에 얼마나 근접하는지를 같은 격자와 같은 엔진에서 측정하였다 (4 장 라). 또한 FlashInfer 자체도 tensor-core 와 CUDA-core 변형 사이의 선택을 사용자에게 맡기므로, 길이 분포를 보는 설정 선택이 필요하다는 점은 두 라이브러리에 공통된다.

다) GPU 성능 분석과 시뮬레이션

(1) GPU 성능 분석

루프라인 모델[15]은 연산 강도, 연산 성능, 메모리 대역폭을 함께 보아 병목을 판단한다. 본 작품은 커널의 최소 이동 바이트와 연산량을 계산하고 실측 대역폭·연산 성능과 비교하며, 디코드 단계를 8 개 연산 클래스로 나누어 각 클래스가 DRAM 또는 텐서 코어 상한의 어느 수준에 있는지 판정한다. 국내의 정종현 외[16]는 마이크로벤치마크로 최신 GPU 의 데이터 캐시 동작을 측정하여 계층별 성능 특성을 분석하였다. 이처럼 실행 시간과 내부 자원 특성을 함께 살펴보는 접근을 분할 수 선택 분석에 적용하였다.

(2) GPU 시뮬레이션과 본 작품의 위치

GPGPU-Sim[6]은 PTX 가상 명령어를 실행하는 상세 GPU 시뮬레이터로, 캐시·인터커넥트·메모리 컨트롤러 등 설계 요소의 영향을 분석한다. Accel-Sim[7]은 실제 GPU 에서 수집한 SASS 명령어 추적과 검증 방법론을 제공한다. 본 작품은 Accel-Sim 의 RTX 4090 구성에서 SM 수, 메모리 대역폭과 L2 크기를 바꾸어 민감도를 분석하고, GPGPU-Sim 에서는 직접 작성한 축소 커널로 부하 불균형의 경향을 확인하였다. 시뮬레이터는 절대 시간이 보정되지 않았으므로 선택 정책에는 쓰지 않고 원인 설명과 민감도 분석에만 사용한다.

Vidur[21]와 같은 서빙 시뮬레이터는 연산자별 프로파일과 예측 모델로 배치 구성·배포 설정을 탐색한다. 본 작품의 성능 모델은 attention 커널 하나의 실행 구성(CTA 별 작업량, 이동 바이트, 연산량)만을 다루는 훨씬 좁은 모델이며, 지연 시간 예측 오차가 아니라 실행 중 후보 선택의 손실(regret)로 평가한다는 점이 다르다.

라) 원안과의 관계 및 접근 방법 비교

원안은 Transformer 와 선택적 상태 공간 모델 Mamba[17]를 입력에 따라 전환하는 시스템이었다. 그러나 두 모델은 구조와 가중치가 달라 생성 결과 자체가 바뀐다. 이에 전환 대상을 같은 모델 안의 attention 실행 설정으로 좁혔다. 표 2 는 관련 연구와 본 작품의 접근을 비교한 것이다.

표 2. 관련 연구와 본 작품의 접근 비교

연구	접근 방법	본 작품과의 관계
Orca[2]·vLLM[3]	반복 단위 스케줄링과 페이지 KV 관리	가변 길이 배치가 발생하는 환경
Sarathi-Serve[9]	프리필 분할과 디코드 동시 처리	배치 구성 수준에서 상호 보완
Splitwise[18]·BurstGPT[19]	실제 서비스의 요청 트레이스 공개	문제의 발생 빈도 추정에 사용
Flash-Decoding[4] FlashAttention-2[5]	KV 분할과 부분 결과 결합, 분할 수 휴리스틱	선택 대상과 기본 휴리스틱
FlashDecoding++[13]	입력·하드웨어별 실행 방식 휴리스틱 선택	설정 선택이라는 접근의 공통점
FlashInfer[14] LeanAttention[20]	커널 내부 부하 분산 스케줄링	커널 수준 해법, 같은 엔진에서 격차 측정
Vidur[21]·Accel-Sim[7]	서빙·GPU 시뮬레이션	원인 설명·민감도 분석에만 사용
본 작품	실측·성능 모델 기반 실행 설정 선택	기존 커널·엔진을 유지하며 TPOT·출력 검증

■ 3. 제안 작품 소개

가) 이론적 배경

(1) 디코드 attention 과 메모리 병목

요청 b 의 헤드는 질의 q 와 길이 L_b 인 키 행렬 K, 값 행렬 V 로 $o = \mathrm{softmax}(qK^{\top}/\sqrt{d})V$ 를 계산한다. Qwen3-4B 는 질의 헤드 32 개가 KV 헤드 8 개를 공유하는 GQA 구조이며 헤드 차원 d 는 128 이다. 한 단계에서 읽는 KV 양은 요청 길이의 합에 비례한다. 디코드에서는 읽는 데이터에 비해 연산량이 작아 메모리 대역폭과 병렬 작업 수가 실행 시간을 좌우한다.

(2) GPU 실행 구조와 부하 불균형

CUDA 커널은 여러 스레드 블록(CTA)의 그리드로 실행된다. 각 CTA 는 한 SM 에 배치되며, 동시에 상주할 수 있는 CTA 수는 레지스터와 공유 메모리 사용량의 제한을 받는다. 점유율(occupancy)은 SM 의 최대 수용 워프 수 대비 활성 워프 수의 비율이다. RTX 4090 은 SM 128 개를 가진다. CTA 별 작업량이 다르면 짧은 작업이 먼저 종료되어 일부 SM 만 동작하는 꼬리 구간이 발생한다.

(3) KV 분할의 이득과 비용

KV 분할은 각 요청·KV 헤드의 캐시를 여러 구간으로 나누어 CTA 들이 부분 attention 을 계산하고, 별도의 결합 커널이 부분 결과를 합치는 방식이다. 본 실험 구성에서 분할 수 S 에 따른 CTA 수는 S×B×H_kv 이다. 분할은 병렬성을 늘리지만 부분 결과를 저장·재읽기하는 트래픽과 결합 비용도 추가한다. 따라서 분할 수가 지나치게 작으면 SM 을 충분히 활용하지 못하고, 지나치게 크면 부가 비용이 증가한다.

attention 이 디코드 단계에서 차지하는 비중을 f, attention 속도 향상을 s 라 하면, 다른 연산 시간이 같을 때 단계 전체의 속도 향상은 Amdahl 의 법칙에 따라 $1/((1-f)+f/s)$이다. 실제 모델 결과에서는 attention 시간 변화로 예측한 단계 시간과 측정값을 비교하여 개선의 원인을 확인한다.

나) 라이브러리 기본 설정과 가변 길이 배치

FlashAttention 2.8.3 은 기본 CTA 수 B·H_kv·m 이 SM 당 CTA 2 개를 가정한 실행 슬롯 수의 80% 이상이면 분할하지 않는다. 그보다 작으면 1~128 범위에서 계산한 최고 활용 효율의 85% 이상을 얻는 가장 작은 분할 수를 선택한다. m 은 질의 행 블록 수이다. 이 규칙과 그리드 구성을 라이브러리 소스 코드에서 이식하여 CTA 별 작업량을 계산하였다(부록 A.1). FlashInfer 의 디코드 래퍼는 GQA 그룹 크기가 4 이상이면 tensor-core 변형을 권장하며, 변형 선택은 길이 분포와 무관하게 생성 시점에 고정된다.

32K 요청 1 개와 512 토큰 요청 31 개에서는 기본 CTA 가 32×8=256 개이므로 기준 204.8 을 넘어 분할 수 1 을 고른다. 그러나 32K 요청을 맡은 8 개 CTA 에 긴 작업이 집중되고 나머지 248 개는 조기에 종료된다. 분할 수 16 에서는 긴 요청의 작업이 128 개 CTA 로 나뉘고 CTA 당 최대 키 수가 2,048 개로 줄어든다.

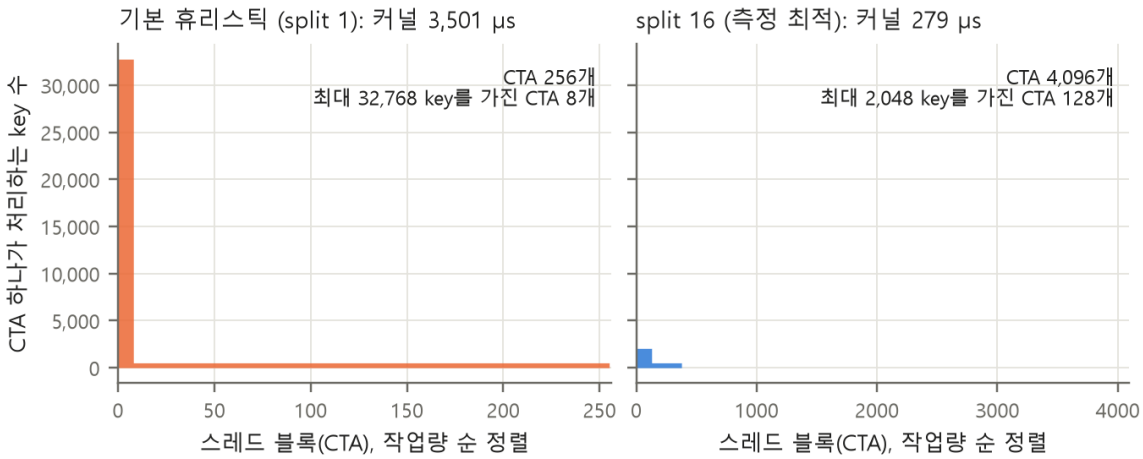


그림 2. 가변 길이 배치에서 분할 수에 따른 CTA 별 키 수

그림 2 는 동일한 요청 구성의 이론적 작업량을 내림차순으로 배열한 것이다. 분할 수 16 에서는 짧은 요청의 대부분 구간이 비어 있으므로 키를 처리하지 않는 CTA 도 포함된다. 즉 CTA 의 총개수를 늘리는 것이 아니라, 긴 요청의 작업을 여러 SM 에 분산하는 것이 개선의 요인이다. 이 조건의 연속 KV 커널 실측 시간은 3,501 µs 에서 279 µs 로 줄었다.

최적 대비 손실은 (선택 커널 시간/동일 조건의 최적 후보 시간)으로 정의한다. 비교 후보와 KV 형식, cold·warm 상태를 고정하여 입력이나 캐시 상태 차이가 선택 정책의 효과로 섞이지 않도록 하였다. 라이브러리를 넘나드는 비교에서는 FlashAttention-2 변형과 FlashInfer 변형을 모두 포함한 전체 최적을 분모로 쓴다.

다) 시스템 구성과 구현

시스템은 Python 패키지 약 9,800 줄, 대시보드·스크립트 약 4,400 줄, 테스트 약 7,900 줄로 구현하였다. 구성 요소는 Parquet·CSV·JSON 결과를 주고받아 GPU 측정과 CPU 분석·재현을 분리한다.

(1) 커널 측정과 실행 구성 분석

분할 수 1, 기본 휴리스틱(num_splits=0), 고정 분할 수 2~128 의 7 종 변형을 등록하여 조건마다 FlashAttention-2 후보 9 개를 비교하고, 같은 페이지 KV 입력에 FlashInfer 의 tensor-core·CUDA-core 변형을 플러그인으로 추가하였다. 연속 KV 와 페이지 KV 를 구분하고, CUDA 이벤트와 torch.profiler 의 CUPTI activity 로 시간·그리드·블록·레지스터·공유 메모리를 기록한다. cold 측정에서는 L2 용량의 4 배 데이터를 쓰는 커널을 먼저 실행한다. 참조 구현과 결과를 비교하고 GPU 사용률·다른 프로세스도 기록한다. 측정 격자는 긴 요청 8K~32K 의 기존 격자에 64K 격자(가변 18 셀, 균일 2 셀)를 더했다.

실행 구성 분석은 CTA 별 키 수와 이동 바이트·연산량을 계산한다. Accel-Sim 은 NVBit 로 수집한 실제 커널 추적을 RTX 4090 구성에서 재실행하고, SM 수·대역폭·L2 를 각각 두 배와 절반으로 바꾸어 자원별 민감도를 분석한다. Nsight Compute 를 사용할 수 없는 공용 서버 환경을 고려하여 하드웨어 카운터에 의존하지 않도록 하였다.

(2) 실측 기반 성능 모델

성능 모델은 CTA 별 키 수를 입력받아 SM 수와 상주 CTA 수로 정한 실행 슬롯에 작업을 배치한다. 동시 실행 CTA 가 대역폭을 나누어 쓰는 이산 사건 시뮬레이션으로 완료 시간을 계산하며, 키당 처리 시간·시작 비용·결합 비용은 균일 길이 실측에 Nelder–Mead 방법으로 맞춘다. NumPy 구현을 C 언어로 재구현하여 첫 선택 비용을 842.6 ms 에서 14.3 ms 로 줄였고, 기존 7 개 조건에서 예측값과 순위의 일치를 확인하였다. 같은 모델은 트레이스 재생에서 매 디코드 단계의 길이 분포를 채점하는 데에도 사용한다.

(3) 디코드 단계의 연산 클래스별 시간 분해

attention 개선이 단계 전체에 얼마나 반영되는지 설명하기 위해, 디코드 단계를 embed, norm, qkv_proj, rope, attention, o_proj, mlp, lm_head 의 8 개 연산 클래스로 나누어 클래스별 GPU 시간을 CUDA 이벤트로 측정한다. 타이머가 없는 대조 실행과 비교하여 측정 오버헤드를 따로 계산하고, 클래스별 최소 이동 바이트와 연산량으로 루프라인 모델[15]의 DRAM·텐서 코어 상한 대비 달성률을 구한다. 달성률이 70% 이상이면 memory-bound 또는 compute-bound 로 판정하고, attention 이 두 상한 모두에 미달하면 병렬성 부족 후보로 표시한다. 이 분해는 Nsight Compute 가 보여 주는 커널 내부 동작을 대체하지 않으며, 하드웨어 카운터 없이 단계 수준의 병목 위치를 설명하는 용도이다.

(4) 공개 트레이스 재생

문제의 발생 빈도를 추정하기 위해 공개 요청 트레이스의 (도착 시각, 프롬프트 토큰 수, 출력 토큰 수)를 실행 엔진과 같은 규칙의 연속 배치 스케줄러에 넣어 GPU 없이 재생한다. FIFO 수용, 배치 상한 64, 요청 수용 시 KV 페이지 예약(KV 예산 10 GiB), 단계 시간 30 ms 고정, 프리필 즉시 처리의 단순화를 두고, 도착 간격을 비율로 압축하여 GPU 한 장의 부하를 만든다. 매 단계의 KV 길이 벡터에 대해 FlashAttention-2 휴리스틱이 고르는 분할 수와 성능 모델이 예측한 최적 분할 수의 시간 비를 손실로 채점한다. 성능 모델의 오차(중앙값 2.3~12.3%)를 고려하여 1.25 배 문턱을 주된 지표로 쓴다.

(5) 재현성 검증과 대시보드

원본 기록과 해시를 결과 아카이브에 저장하고 kernelscope verify 로 문서의 주요 수치 531 개를 다시 계산한다. 대시보드는 결론·토큰 생성 비교·원인·선택·검증의 다섯 장면으로 구성된 시연 페이지와, 병목 진단·커널 지도·하드웨어 조건 변경·실제 생성 결과·정책 비교의 분석 탭 5 개를 제공하며 실측과 예측을 구분한다. 토큰 생성 비교는 같은 GPU 에서 순차로 측정한 기록을 타임스탬프대로 동시에 재생하는 방식이며, 두 정책을 동시에 실행하지 않는다. 두 기능 모두 GPU 없이 실행된다.

라) 선택 정책, 실행 엔진, 출력 검증 절차

정책은 매 단계 요청 길이 목록을 받아 분할 수를 반환하며, 라이브러리 교체 정책은 커널 변형도 함께 반환한다. 구현한 정책은 표 3 과 같다. 테이블·모델·하이브리드 정책은 256 토큰 페이지 단위로 양자화한 길이 목록을 캐시 키로 사용하고, 정책 계산 CPU 시간을 별도로 기록한다. 정책 캐시를 측정 실행마다 초기화하는 설정과 실행 간에 유지하는 설정을 모두 지원하여 첫 결정 비용의 영향을 분리한다.

표 3. 구현한 실행 설정 선택 정책

정책	선택 방법
기본 휴리스틱	num_splits=0 으로 라이브러리 규칙 사용
고정값	항상 같은 분할 수 사용(예: 8)
조회 테이블	같은 헤드 구성에서 배치 크기·최대 길이·평균 길이의 로그 거리가 가장 가까운 실측 조건의 최적 분할 수 사용
성능 모델	FlashAttention-2 후보 9 개의 실행 시간을 예측해 최소값 선택
하이브리드	성능 모델의 순위를 따르되, 가장 가까운 실측 조건의 최적 변형이 모델의 최선 예측 시간의 20% 이내로 예측되면 그 변형 선택(부록 A.3)
FlashInfer 백엔드	분할 수 대신 FlashInfer tensor-core 또는 CUDA-core 변형을 고정 사용, 매 단계 plan 비용을 정책 비용에 기록
라이브러리 교체 테이블	가장 가까운 실측 조건에서 FlashAttention-2 최적 분할과 FlashInfer 두 변형의 층 수×커널 시간(선택적으로 plan 비용 가산)을 비교해 최소 후보 선택

실행 엔진은 safetensors 의 bf16 가중치를 읽고 페이지 KV 를 관리하며, flash_attn_with_kvcache 에 선택한 분할 수를 전달한다. FlashInfer 백엔드는 디코드 단계에서 같은 자리에 들어가며, 단계마다 plan 을 한 번 호출하고 층마다 run 을 호출한다(부록 A.4). 프리필은 두 경우 모두 FlashAttention-2 를 사용한다. 시나리오의 논리적 도착 단계에 따라 요청을 배치에 추가하고 탐욕적 디코딩으로 고정 길이를 생성한다 . attention·전체 단계 시간과 토큰 생성 시각을 기록하고, 엔진의 수치적 정확성은 독립된 Hugging Face transformers 구현과 비교하였다.

출력 검증은 세 단계로 수행한다. 첫째, 자유 생성에서 기준 정책(기본 휴리스틱)과 후보 정책의 토큰 이력을 비교하여 요청별 첫 불일치 위치를 분기 지점으로 센다. 탐욕적 디코딩에서는 한 토큰이 달라지면 이후 토큰이 연쇄적으로 달라지므로 불일치 토큰 수가 아니라 분기 지점 수를 비교 단위로 쓴다 . 둘째, 기준 정책의 토큰 이력을 후보 정책에 입력하는 teacher forcing 진단으로 분기 지점의 로짓을 기록하고, 기준 1 위 로짓과 후보가 고른 토큰의 로짓 차이를 기준 1 위 로짓 크기의 bf16 ULP 단위로 분류한다(1 ULP 이내, 2 ULP 이내, 그 이상). 셋째, 2 ULP 를 넘는 사건은 사건이 난 단계의 KV 캐시 상태를 기준 이력으로 재구성한 뒤 36 개 층 각각의 attention 출력을 기준 설정·후보 설정·fp32 참조로 비교하는 커널 수준 수치 검사로 구현 오류 여부를 판정한다. 층별 차이가 그 층 최대 출력의 bf16 1 ULP 이내이면 커널은 같은 입력에서 같은 결과를 낸 것으로 보고, 같은 상태에서 후보 설정으로 계산한 한 단계의 argmax 변화 여부를 함께 기록한다.

■ 4. 구현 및 결과분석

가) 실험 환경과 구현 현황

실험 환경은 표 4 와 같다. 8 개 측정 집합에서 워크로드, 캐시 상태, 후보 커널의 조합 6,801 개를 측정한 기존 결과에 FlashInfer 두 변형 422 개 조건과 64K 격자 220 개 조건을 더해 총 7,443 개 조건을 측정하였으며 오류는 없었다. GPU 를 숨긴 CPU 환경에서 테스트 667 개가 통과하였고(5 개 생략), 문서 수치 재계산 검증 531 개가 통과하였다.

표 4. 실험 환경

항목	내용
GPU	RTX 4090, SM 128 개, L2 72 MiB 실측 대역폭 952.6 GB/s, 텐서 코어 168.6 TFLOPS
소프트웨어	PyTorch 2.8.0(CUDA 12.8 빌드), CUDA 12.9, 드라이버 580.95.05 FlashAttention 2.8.3.post1, flashinfer-python 0.6.13(cu128 사전 빌드 커널)
모델·시뮬레이터	Qwen3-4B-Instruct-2507, DeepSeek-R1-Distill-Llama-8B Accel-Sim v2.0.0, NVBit 1.8, GPGPU-Sim 4.2; 운영 엔진 재현은 별도 환경의 vLLM 0.31.0
측정 범위	균일: B=1~64, L=512~64K 가변: B=16~64, 긴 요청 1·2·4 개, 긴 길이 8K·16K·32K·64K, 짧은 길이 512·1K·2K, 헤드 구성 2 종
트레이스	Azure LLM Inference 2023(채팅 19,366 건, 코드 8,819 건), BurstGPT v2(7 일 17,893 건, GPT-4 14 일 14,265 건)

나) 커널 수준 측정 결과

표 5 는 기본 휴리스틱의 최적 분할 수 대비 손실을 요약한 것으로, cold 상태의 결과이며 기본 헤드 구성 S1 은 질의 32 개·KV 8 개이다. 균일 길이에서는 휴리스틱이 최적에 가까웠지만 가변 길이에서는 손실이 크게 증가했다. 최악 연속 KV 조건(32K×1+512×31)은 12.53 배 느렸다. 같은 입력의 페이지 KV 는 3.84 배로 차이가 작았으며, 실제 엔진이 페이지 KV 를 사용하므로 전체 모델의 개선은 이 경로를 기준으로 해석한다.

표 5. 기본 휴리스틱의 최적 분할 수 대비 손실 요약(FlashAttention-2 후보 기준)

측정 집합	조건 수	중앙값	최댓값	10% 초과
균일·연속 KV	49	0.7%	5.7%	0
균일·페이지 KV	49	1.2%	10.0%	0
가변·연속 KV	162	172.4%	1152.7%	124
가변·페이지 KV	162	43.6%	298.1%	127
가변·페이지 KV(S2)	135	47.2%	432.3%	102

같은 페이지 KV 격자에서 FlashInfer 의 두 변형을 함께 측정하고, 모든 후보 중 전체 최적을 분모로 각 기본 설정의 손실을 구한 결과가 표 6 이다. 가변 길이 32K 격자 162 셀에서 FlashInfer 의 tensor-core 기본 변형은 중앙값 1.02 배로 대체로 양호하지만 90 퍼센타일 1.49 배, 최악 2.49 배이며 36 셀이 1.25 배를 넘었다. 균일 길이에서는 두 라이브러리의 기본 설정 모두 1.2 배 이내였다. 즉 두 라이브러리의 기본 설정은 가변 길이 배치에서 같은 방향으로 틀린다. 반면 FlashInfer CUDA-core 변형(최악 1.01 배)과 FlashAttention-2 의 분할 수만 고르는 본 작품의 선택기(최악 1.06 배)는 모두 전체 최적에 근접하였다. 162 셀 중 전체 최적 커널은 FlashInfer CUDA-core 139 셀, tensor-core 21 셀, FlashAttention-2 2 셀로 두 변형 중 빠른 쪽의 커널 자체는 FlashAttention-2 의 최적 분할보다 조금 빠르지만(중앙값 2.5%), 이득의 대부분은 분할 수, 즉 부하 분배에서 나온다.

표 6. 두 라이브러리 기본 설정의 전체 최적 대비 손실(페이지 KV, cold)

격자	셀	기본 설정	중앙값	p90	최악	1.25 배 초과
32K 가변	162	FlashAttention-2 휴리스틱	1.457 배	2.519 배	4.106 배	118 셀
		FlashInfer tensor-core	1.017 배	1.489 배	2.485 배	36 셀
		FlashInfer CUDA-core	1.000 배	1.003 배	1.014 배	0
		FA2 분할 수만 선택(본 작품)	1.025 배	1.043 배	1.063 배	0
32K 균일	49	FlashAttention-2 휴리스틱	1.025 배	1.092 배	1.171 배	0
		FlashInfer tensor-core	1.000 배	1.035 배	1.053 배	0
64K 가변	18	FlashAttention-2 휴리스틱	2.861 배	4.521 배	5.265 배	18 셀
		FlashInfer tensor-core	1.634 배	2.463 배	2.989 배	12 셀
		FA2 분할 수만 선택(본 작품)	1.023 배	1.044 배	1.049 배	0

긴 요청의 길이에 따른 추세는 그림 3 과 같다. 배치 크기와 짧은 요청 길이가 같은 18 셀을 길이별로 비교하면, 기본 휴리스틱의 최적 분할 수 대비 손실 중앙값은 8K 1.55 배, 16K 1.93 배, 32K 2.56 배, 64K 2.84 배로 단조 증가하고 최악은 5.18 배에 이른다. FlashInfer tensor-core 기본 변형도 1.13 배에서 1.63 배로 커지며, CUDA-core 변형은 길이와 무관하게 최적에 붙어 있다. 64K 대표 셀(65,536+512×31)에서 휴리스틱은 1,957 μs, 최적 분할 수 16 은 422 μs, FlashInfer CUDA-core 는 415 μs, tensor-core 는 1,240 μs 였다. 긴 요청이 길어질수록 그 CTA 하나의 작업이 배치 전체를 지배하므로, 문맥이 길어지는 워크로드일수록 문제가 커진다.

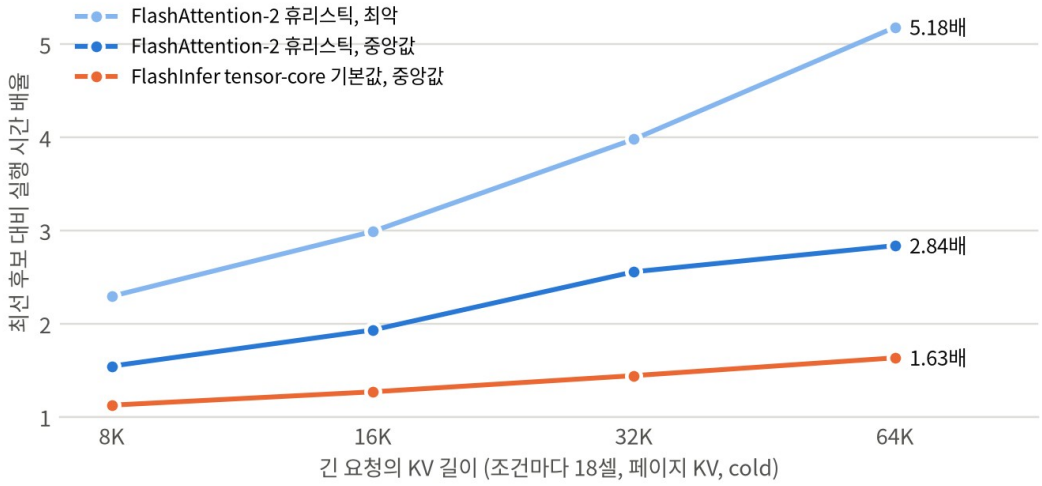


그림 3. 긴 요청 길이에 따른 기본 설정의 최적 대비 손실(조건이 같은 18 셀, 페이지 KV)

다) 문제의 발생 빈도: 공개 트레이스 재생

표 7 과 그림 4 는 공개 트레이스를 재생하여 매 디코드 단계의 손실을 채점한 결과이다. Azure 채팅 트레이스를 RTX 4090 한 장이 받는 부하(도착 간격 원본, 평균 배치 34.8)로 재생하면 단계의 89.5%가 분할 없음 조건(배치 26 이상)에 해당하고, 13.5%가 attention 시간을 1.25 배 이상 잃으며, 전체 attention 시간 기준으로는 최적 분할 수를 고르는 선택기가 10.6%를 절감한다. 코드 트레이스는 평균 배치 27 에서 14.1%였다. 프롬프트 중앙값이 502 토큰인 BurstGPT 에서는 어느 부하에서도 1.5% 이하였다.

표 7. 공개 트레이스 재생에서 기본 휴리스틱의 손실(성능 모델 채점, 1.25 배 문턱)

트레이스·부하	평균 배치	분할 없음 단계	손실 ⩾1.25 배 단계	attention 시간 비
Azure 채팅 ×0.5	17.4	6.2%	2.4%	1.054
Azure 채팅 ×1 (117,035 단계)	34.8	89.5%	13.5%	1.119
Azure 채팅 ×2	47.0	98.8%	4.1%	1.092
Azure 코드 ×16	27.4	70.1%	14.1%	1.136
BurstGPT 7 일 ×100	26.5	50.1%	0.3%	1.018
BurstGPT GPT-4 14 일 ×300	29.6	66.7%	1.5%	1.030
what-if: 채팅 ×1, 프롬프트 ×2, KV 10 GiB	27.1	70.8%	25.7%	1.156
what-if: 채팅 ×1, 프롬프트 ×4, KV 10 GiB	14.6	0.1%	2.9%	1.064
what-if: 채팅 ×1, 프롬프트 ×4, KV 436K 토큰	34.8	89.3%	39.5%	1.222

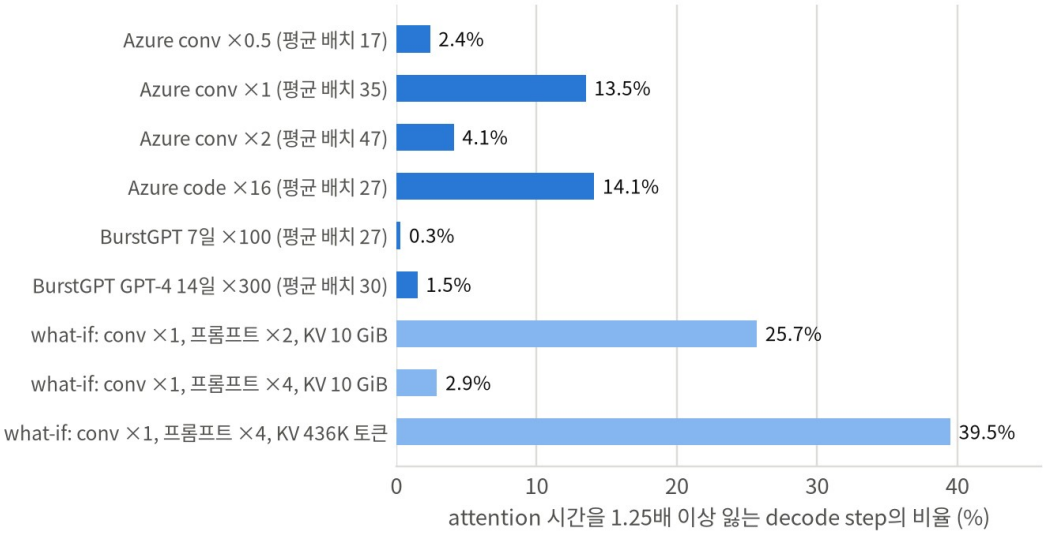


그림 4. 트레이스별 손실 1.25 배 이상 디코드 단계 비율(진한 파랑: 공개 트레이스, 연한 파랑: what-if)

빈도는 배치 크기와 길이 편차의 곱으로 정해진다. 부하가 낮아 배치가 26 미만이면 휴리스틱이 스스로 분할하므로 손실이 2~5%로 줄고, 배치가 47 에 이르면 CTA 수가 많아져 긴 요청 하나의 비중이 희석되어 다시 4.1%로 준다. 프롬프트를 2 배로 늘리면 손실 단계가 25.7%로 커지지만, 4 배로 늘리면 10 GiB KV 예산 안에서 배치가 14.6 으로 줄어 손실이 사라진다. KV 예산을 436K 토큰(80 GB 급 GPU)으로 두면 배치가 유지되어 39.5%까지 커진다. 즉 긴 문맥 서빙에서 이 문제가 커지는 조건은 긴 프롬프트와 배치를 26 이상으로 유지할 KV 예산이 함께 있을 때이다. 이 결과는 성능 모델의 예측이며 실측이 아니고, 단계 시간 고정과 즉시 프리필의 단순화를 포함한다. 또한 어느 트레이스도 32K 요청을 담고 있지 않으므로 본 작품의 최악 조건은 트레이스에 나타나지 않는 극단이다.

라) 실제 LLM 생성 결과

(1) TPOT 과 출력 동일성

Qwen3-4B 에서 균일(512×32), 가변(32K×1+512×31), 요청 도착 시나리오를 비교하였다. 앞의 두 조건은 요청당 64 토큰을 생성한다. 요청 도착은 16K×1+512×15 로 시작해 8·24 단계에 짧은 요청 16 개씩을 추가한다. 9 월의 조회 테이블 캠페인은 정책별로 실행 순서를 바꾸어 5 회, 10 월의 하이브리드 정책 캠페인과 64K 캠페인은 3 회 반복하였다. 결과는 표 8 과 같다.

표 8. Qwen3-4B 의 TPOT 과 출력 동일성

시나리오	정책	기본 TPOT	정책 TPOT	개선	토큰
균일 길이	테이블(5 회)	27.42 ms	27.50 ms	1.00 배	일치
가변 길이 32K	테이블(5 회)	61.06 ms	33.80 ms	1.81 배	일치
요청 도착	테이블(5 회)	59.09 ms	50.34 ms	1.17 배	일치
균일 길이	하이브리드(3 회)	27.63 ms	27.83 ms	0.99 배	일치
가변 길이 32K	하이브리드(3 회)	61.11 ms	34.09 ms	1.79 배	일치
요청 도착	하이브리드(3 회)	59.33 ms	51.79 ms	1.15 배	일치

시나리오	정책	기본 TPOT	정책 TPOT	개선	토큰
가변 길이 64K	테이블(3 회)	96.93 ms	40.06 ms	2.42 배	분기 1 건
가변 길이 64K	하이브리드(3 회)	96.93 ms	39.35 ms	2.46 배	분기 1 건

가변 길이 32K 에서 테이블 정책은 단계당 attention 시간을 36.87 ms 에서 9.24 ms 로 줄였다. attention 시간 차이만 반영한 예측 단계 시간과 실제 측정값의 차이는 1.5%로, 개선이 attention 커널 선택에 기인함을 확인하였다. 요청 도착에서는 긴 요청 비중이 작아 개선 폭이 1.17 배였다. 하이브리드 정책은 세 조건이 실측 테이블의 셀과 겹쳐 567 단계 모두 테이블과 같은 분할 수를 골랐으므로 TPOT 차이는 선택 비용(단계당 0.25~0.75 ms, 실행당 1~5 회의 첫 결정 비용)뿐이며, 9 월 캠페인의 휴리스틱·테이블 TPOT 이 1.25% 이내로 재현되었다. 64K 조건에서는 측정 테이블에 64K 셀이 없어 테이블 정책은 가장 가까운 32K 셀의 분할 수를 외삽하여 썼고, 성능 모델은 실행 구성에서 예측하였다. 세 정책이 모두 분할 수 8 을 골라 TPOT 을 2.42~2.46 배 개선하였고, 휴리스틱에서 attention 이 단계의 82.8%를 차지하므로 32K 보다 상한이 높았다.

균일 길이는 기본 휴리스틱이 이미 최적에 가까워 이득이 없었다. 고정 분할 수 8 은 가변 길이에서 TPOT 33.39 ms 를 얻었지만 균일 길이에서는 토큰이 달라졌으며, 이 사례는 (5)의 출력 동일성 분석에서 대조군으로 사용한다. 초기 NumPy 성능 모델은 첫 선택의 계산 비용 때문에 균일 길이와 요청 도착에서 TPOT 이 오히려 증가하였고, 이에 따라 모델 계산을 C 언어로 재구현하여 첫 선택 비용을 약 59 배 줄였다. 후속 실험(두 모델×자연어 입력 조건 3 개×시드 2 개×정책 3 개×반복 3 회, 총 108 회)에서 성능 모델은 가변 길이(12K×1+384×27)의 TPOT 을 Qwen3-4B 에서 1.282~1.290 배, Llama 계열 8B 에서 1.184~1.188 배 개선했고 토큰도 같았다.

(2) 디코드 단계의 연산 클래스별 분해

그림 5 는 가변 길이 32K 조건에서 디코드 단계를 8 개 연산 클래스로 분해한 결과이다. 기본 휴리스틱에서는 attention 이 단계의 70.9%를 차지하고 DRAM 상한의 21%에 머물러 병렬성 부족으로 판정되었으며, 테이블 정책에서는 37.5%로 낮아지고 DRAM 상한의 83%에 도달하여 memory-bound 로 바뀌었다. 나머지 GEMM 클래스(qkv_proj·o_proj·mlp·lm_head)는 두 정책 모두 DRAM 상한의 63~90%에 있어 커널 선택으로 더 줄일 여지가 작다. 균일 길이에서는 attention 비중이 19.2%에 불과하고 두 정책이 같은 변형을 고르므로 단계 시간 차이가 없다. 이는 균일 배치에서 이득이 없는 이유의 실측 근거이다. attention 만 최적으로 바꿨을 때의 Amdahl 추정 2.10 배와 실측 단계 시간 비 2.14 배는 1.8% 차이였다. 추정은 가장 가까운 실측 셀의 cold 손실로 계산한 값이라 실측이 조금 위에 있으며, 엄밀한 상한으로 주장하지 않는다. 측정되지 않은 시간은 최대 2.8%, 타이머 오버헤드는 최대 4.2%였다.

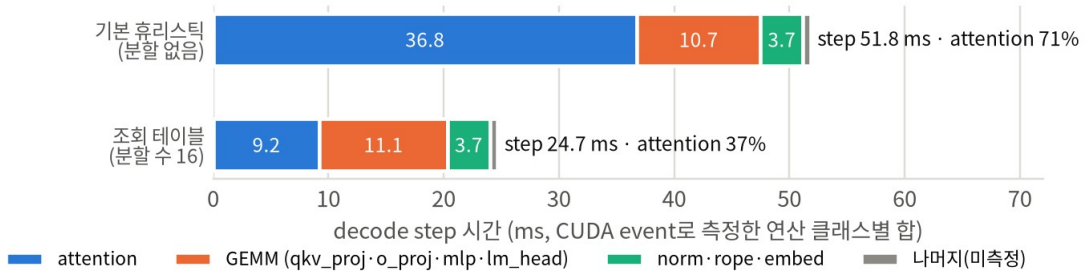


그림 5. 가변 길이 32K 조건의 디코드 단계 연산 클래스별 시간(Qwen3-4B, CUDA 이벤트)

(3) 하이브리드 정책이 테이블과 다르게 선택하는 조건

요청한 세 조건에서는 하이브리드 정책이 테이블과 같은 선택을 했으므로, GPU 없이 기록된 길이로 정책을 재생하여 두 정책이 갈리는 조건(자연어 입력의 요청 도착, 12K 토큰 요청 1 개와 384 토큰 요청 27 개가 7 차례에 걸쳐 도착)을 찾아 추가로 측정하였다. 결과는 표 9 와 같다. 하이브리드 정책은 55 단계 중 12 단계에서 성능 모델의 순위를 따라 attention 시간을 테이블(3.31 ms)보다 짧은 3.00 ms 로 줄였다. 그러나 측정 실행마다 정책 캐시를 초기화하는 설정에서는 실행당 13 회의 첫 결정 비용(단계당 1.56 ms)이 attention 에서 얻은 0.3 ms 를 넘어 TPOT 개선이 1.02 배에 그쳤다. 정책 캐시를 실행 간에 유지하는 설정으로 재측정하면 선택 비용이 단계당 15 µs 로 떨어져 하이브리드 1.090 배, 모델 1.091 배, 테이블 1.086 배로 순위가 attention 시간의 차이와 일치하였다. 두 설정은 실제 서빙의 양 끝이며, 요청 구성이 매 단계 바뀌는 서비스는 전자에, 같은 구성이 되풀이되는 부하는 후자에 가깝다.

표 9. 자연어 요청 도착 조건에서의 정책 비교(Qwen3-4B, 3 회)

정책	attention ms/단계	TPOT(캐시 초기화)	개선	TPOT(캐시 유지)	개선
휴리스틱	4.85	34.44 ms	1.000	34.46 ms	1.000
테이블	3.31	31.74 ms	1.085 배	31.73 ms	1.086 배
하이브리드	3.00	33.77 ms	1.020 배	31.62 ms	1.090 배
성능 모델	2.93	33.49 ms	1.028 배	31.58 ms	1.091 배

(4) FlashInfer 와의 비교: 커널 교체 대 외부 설정 선택

커널 격자에서 FlashInfer CUDA-core 변형은 가변 길이 162 셀에서 최적 FlashAttention-2 분할 수 대비 중앙값 0.977 배, 최대 1.003 배로 같은 수준이었고, tensor-core 변형은 최악 셀에서 2.40 배 느렸다. 최악 셀(32K×1+512×31, 페이지 KV)의 실측은 휴리스틱 1,072 µs, 최적 분할 수 16 은 279 µs, FlashInfer CUDA-core 270 µs, tensor-core 671 µs 였다. 같은 결론을 전체 모델에서 확인하기 위해 FlashInfer 를 실행 엔진에 통합하고 세 시나리오를 같은 규약(정책 캐시 유지, 3 회 반복)으로 비교하였다. 결과는 그림 6 과 표 10 과 같다.

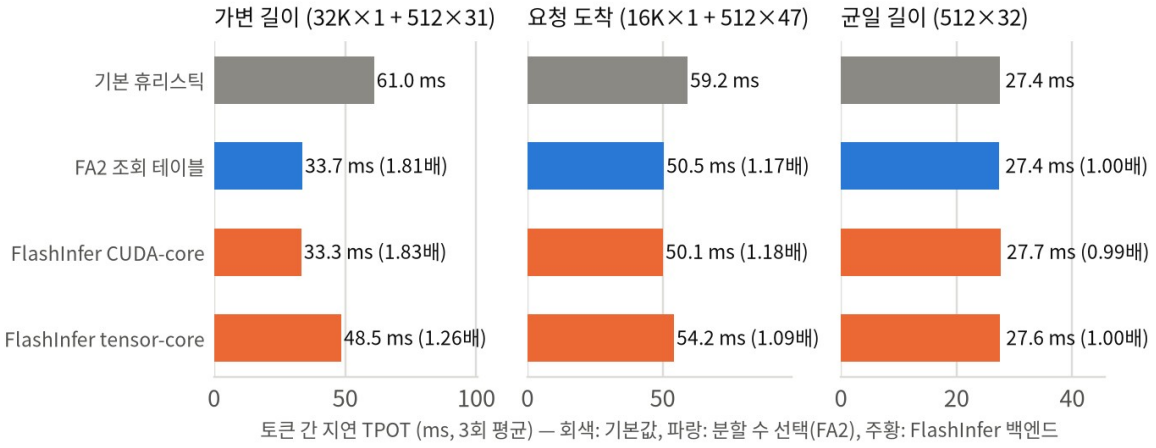


그림 6. 같은 엔진에서의 정책별 TPOT(Qwen3-4B, 3 회 평균)

표 10. 가변 길이 32K 조건의 엔진 수준 비교(Qwen3-4B)

정책	TPOT	개선	attention ms/단계	attention 비중	정책 비용 µs/단계
기본 휴리스틱	60.99 ms	1.000	36.78	71.9%	3.7
FA2 조회 테이블	33.70 ms	1.810 배	9.23	38.5%	6.7
FlashInfer CUDA-core	33.28 ms	1.833 배	8.69	36.8%	370.8 (plan)
FlashInfer tensor-core	48.51 ms	1.257 배	23.87	61.6%	432.4 (plan)
라이브러리 교체 테이블	33.26 ms	1.831 배	8.69	36.9%	382.2 (plan 포함)
라이브러리 교체 테이블 (plan 비용 가산)	33.71 ms	1.806 배	9.23	38.6%	7.1

가변 길이에서 CUDA-core 변형으로 커널을 교체한 정책(1.833 배)과 기존 커널의 분할 수만 선택한 테이블 정책(1.810 배)은 같은 수준이었다. attention 시간은 FlashInfer 쪽이 단계당 0.54 ms 짧지만 매 단계 plan 비용 0.37 ms 가 더해진다. FlashInfer 의 기본 변형(tensor-core)은 1.257 배에 그쳐 테이블 정책이 얻은 이득의 대부분을 잃었고, attention 이 단계의 61.6%를 차지하였다. 균일 길이에서는 네 정책이 ±1% 안에 있었으며, FlashInfer 커널이 조금 빠른 만큼을 plan 비용(단계당 291~338 µs)이 상쇄하였다. 요청 도착에서도 순서는 같았다(CUDA-core 1.181 배, 테이블 1.173 배, tensor-core 1.092 배).

정책이 분할 수와 커널 변형을 함께 고르는 라이브러리 교체 테이블은 별도 캠페인으로 측정하였다(표 10 의 아래 두 행, 같은 캠페인의 휴리스틱 60.90 ms·테이블 33.64 ms 기준). 가변 길이의 모든 단계에서 CUDA-core 를 선택하여 테이블보다 1.15% 빨랐고(3 회 모두), 요청 도착에서는 0.36%로 같은 선택을 한 두 정책 간 편차(0.19%)와 구분되지 않았으며, 균일 길이에서는 FlashAttention-2 기본값을 골라 동률이었다. 후보 비용에 plan 비용 371 µs 를 더해 비교하면 가변 길이에서 분할 수 16 으로 돌아가 테이블과 같은 선택을 재현하였다. 즉 분할 수를 외부에서 선택하는 것으로 커널 교체 이득의 대부분을 얻으며, 라이브러리까지 함께 고르는 추가 이득은 이 GPU 에서 약 1%이고 그 대가로 FlashAttention-2 기준의 출력 동일성을 잃는다. 이득의 한계를 정하는 것은 커널 이득과 같은 자릿수인 단계당 plan 비용이며, 한 번 계획한 결과를 단계 사이에 재사용하는 방법은 시도하지 않았다.

(5) 출력 동일성 분석

표 11 은 캠페인별 분기 지점과 ULP 분류 결과이다. 가변 길이 32K·균일·요청 도착의 테이블·하이브리드 정책은 분기 0 건이었고 teacher forcing 진단에서도 argmax 가 뒤바뀐 위치가 없었다. 자연어 요청 도착에서는 정책별 2 건의 분기가 있었고 6 건 모두 기준 1 위 로짓(31.25~38.75)과 후보 토큰의 로짓 차이가 1 ULP(0.125~0.25) 이내였다. 고정 분할 수 8 대조군(균일 길이)의 분기 6 건 중 5 건은 1 ULP 이내였으나 1 건은 70 ULP 의 차이를 보였다. 64K 조건의 분기 1 건은 11 ULP 였다. FlashInfer 정책은 다른 커널을 쓰므로 기준과 토큰이 같지 않으며, 시나리오당 4~7 건의 분기 중 대부분이 1~2 ULP 이내였고, 가변 길이와 균일 길이에서 각 1 건이 대조군과 같은 위치·같은 토큰의 70 ULP 사건이었다(요청 도착 시나리오에는 그 위치가 없다). 모든 사건은 3 회 반복에서 같은 위치에서 재현되었다.

표 11. 분기 지점과 ULP 분류

캠페인·정책	분기 지점	1 ULP 이내	2 ULP 이내	그 이상	커널 수준 검사
32K·균일·요청 도착 × 테이블·하이브리드(9 회)	0	–	–	–	–
자연어 요청 도착 × 테이블·하이브리드·모델	2 / 정책	6	0	0	–
대조군: 균일 × 고정 분할 8	6	5	0	1 (70 ULP)	층별 ⩽1 ULP, argmax 유지
64K × 테이블·모델·하이브리드	1	0	0	1 (11 ULP)	층별 ⩽1 ULP, 한 단계에서 argmax 변경
FlashInfer CUDA-core (32K·균일·도착)	4·7·6	3·4·6	0·2·0	1·1·0 (70 ULP)	층별 ⩽1 ULP, argmax 유지
FlashInfer tensor-core (32K·균일·도착)	4·4·7	3·3·7	0·0·0	1·1·0 (70 ULP)	층별 ⩽1 ULP, argmax 유지

2 ULP 를 넘은 사건은 위치 기준으로 두 개(대조군·FlashInfer 에서 같은 위치의 70 ULP 사건, 64K 의 11 ULP 사건)이며, 대조군·FlashInfer 두 변형·64K 의 네 차례 커널 수준 수치 검사를 모두 통과하였다. 1~2 ULP 이내의 사건은 검사하지 않았다. 대조군 사건에서는 같은 KV 캐시 상태에서 분할 수 1 과 8 의 36 개 층 attention 출력이 fp32 참조와 최대 0.134 차이(bf16 출력 반올림, 두 설정에서 동일)였고 두 설정 간 차이는 최대 0.0625 로 1 ULP 였으며, 그 단계만 분할 수 8 로 계산한 로짓의 argmax 는 32 개 요청 모두 유지되었다. 따라서 70 ULP 의 차이는 고정 분할 실행이 40 단계 동안 자기 KV 캐시에 쌓은 반올림 차이가 증폭된 것이며, 앞 문맥의 어느 자리를 이어 쓸지 결정하는 attention 점수의 동점에서 갈린 것으로 해석된다. FlashInfer 두 변형의 같은 사건도 층별 차이가 최대 0.25 로 해당 층 최대 출력(33.1)의 1 ULP 이고 argmax 가 유지되었다. 64K 사건은 층별 차이가 1 ULP 이내라는 점은 같지만, 같은 KV 상태에서 분할 수 8 로 계산한 한 단계만으로도 64K 요청의 argmax 가 바뀌었다(참조 1 위 토큰과 후보 토큰의 로짓 차이 0.69, 분할 수 1 과 8 의 로짓 최대 차이 8.99). 65K 무작위 토큰 문맥에서 참조 분포가 평평하여 한 단계의 층별 반올림만으로 순위가 바뀐 경우이며, 커널은 같은 결과를 내지만 긴 무작위 문맥에서 모델 출력 자체가 그만큼 불안정하다는 뜻이다 . 따라서 "분할 수가 달라도 답이 바뀌지 않는다"고 단정하지 않고, 검증한 조건에서의 분기 수와 분류를 보고한다.

마) 성능 모델 검증과 하이브리드 정책

성능 모델과 하이브리드 정책의 오프라인 검증 결과는 표 12 와 같다. V1 은 균일 길이 보간, V5 는 미학습 헤드 구성, V6 는 가변 길이 검증 집합이다. 시간 예측은 각 집합의 목표를 통과했고 손실 중앙값도 0.0%였으나, 모델 단독의 최악 손실은 기준 15%를 넘었다. 예측 오차가 작더라도 후보 간 순위가 역전될 수 있기 때문이다.

표 12. 성능 모델과 하이브리드 정책의 오프라인 검증(leave-one-out)

집합·캐시	조건 수	예측 오차 중앙값	모델 최악 손실	하이브리드 최악 손실
V1·cold	594	9.0%	24.3%	1.97%

집합·캐시	조건 수	예측 오차 중앙값	모델 최악 손실	하이브리드 최악 손실
V1·warm	594	2.3%	41.7%	41.7%
V5·cold	882	12.3%	66.3%	7.54%
V6·cold	1,971	11.1%	24.5%	10.90%

하이브리드 정책은 가까운 실측 조건의 최적 변형을 모델의 최선 예측 시간보다 20% 이내로 예측하면 그 변형을 선택한다. 각 조건을 제외하고 테이블을 만드는 leave-one-out 평가에서 cold 집합의 최악 손실이 모두 15% 이내였다. warm 상태의 V1 에서는 하이브리드 정책을 포함한 모든 정책의 최악 손실이 41.7%로 기준을 충족하지 못하였다. 서빙 엔진은 cold 상태를 가정하므로 본 결과의 결론에는 영향이 없으나, 성능 모델의 warm 대역폭 항은 개선 과제로 남는다. 실제 생성에서의 검증 결과는 라)의 (1)과 (3)에 보고하였다.

바) 시뮬레이터, 운영 엔진 재현, 재현성 검증

Accel-Sim 은 FlashAttention-2 의 B=1 조건에서 실측 시간의 1.04~1.06 배였으나, Flash-Decoding 과 큰 배치에서는 1.5~3.1 배 차이를 보였다. L2 파티션 해시 설정에서 48 개 슬라이스 중 16 개에 읽기 요청이 약 2 배 집중되는 편향을 확인하였다. 현재는 절대 시간보다 자원별 민감도 분석에 사용한다. B=16 에서 대역폭을 절반으로 줄이면 시간이 약 97~102% 늘어나는 경향을 확인하였다.

GPGPU-Sim 의 SM7_QV100 구성에서는 직접 작성한 fp32 축소 커널로 기전을 검증하였다. 가변 길이 배치는 분할 수 1 에서 균일 배치보다 8.5 배 오래 걸렸고, 분할 수 16 에서 7.23 배 빨라졌다. 균일 배치는 8 에서 2.25 배로 포화한 뒤 결합 비용으로 느려졌다. 이는 실측과 같은 경향이지만 실제 FlashAttention-2 의 절대 시간이나 최적 분할 수를 재현한 결과는 아니다.

운영 엔진에서의 재현은 vLLM 0.31.0 을 별도 환경에 두고 같은 모델에 같은 가변 길이·균일 배치를 넣어 생성 길이 4 와 64 의 소요 시간 차로 단계당 디코드 시간을 추정하였다(표 13). 가변 길이 배치는 균일 배치보다 단계당 5.46 배 느렸지만 그 손실은 attention 백엔드(FLASH_ATTN 대 FLASHINFER 0.997 배), CUDA Graph 모드, KV 블록 크기와 무관하였다. 본 작품의 엔진에서 같은 배치의 단계 시간이 휴리스틱 51 ms, 테이블 24 ms 인 것에 비해 vLLM 의 79~81 ms 는 attention 커널 선택으로 설명되지 않는 다른 비용이 지배한다. 즉 가변 길이 배치가 느리다는 증상은 재현되지만, 그 원인이 분할 수 휴리스틱이라는 본 작품의 기전은 이 엔진 버전에서 확인되지 않았으며, 디코드 단계의 프로파일링으로 attention 시간을 분리하는 일이 남아 있다.

표 13. vLLM 0.31.0 에서의 단계당 디코드 시간 추정(Qwen3-4B, 3 회 중앙값)

실행 모드	KV 블록	백엔드	가변 길이 32K×1+512×31	균일 512×32
eager	16	FLASH_ATTN	80.52 ms	14.74 ms
eager	16	FLASHINFER	80.27 ms	14.52 ms
CUDA Graph	16	FLASH_ATTN	78.94 ms	14.12 ms
CUDA Graph	16	FLASHINFER	79.37 ms	13.90 ms
eager	256	FLASH_ATTN	81.27 ms	14.69 ms
eager	256	FLASHINFER	81.32 ms	14.55 ms

GPU 가 없는 환경에서 커널·생성·후속 실험·선택 비용·성능 모델·연산 분해·하이브리드 실생성·분기 분류·트레이스 재생·두 라이브러리 기본 설정·64K·커널 수준 검사·FlashInfer 통합·라이브러리 교체 테이블의 주요 수치 531 개를 원본 기록으로 재계산했고 모두 일치하였다. 검증은 데이터와 문서의 값이 달라지는지도 확인한다. CPU 시연은 스케줄링·토큰 비교 로직을 검증할 수 있으나 분할에 따른 GPU 성능 차이는 재현하지 않는다.

사) 한계와 보완 계획

실험은 RTX 4090 한 종류, FlashAttention 2.8.x, 통제된 합성·자연어 입력에 한정된다. 트레이스 재생은 성능 모델의 예측이며 단계 시간 고정과 즉시 프리필을 가정한다. 반복은 3~5 회이므로 같은 선택을 한 정책 사이의 1% 미만 차이는 구분하지 못한다. vLLM 에서 관찰된 가변 길이 손실의 원인, FlashInfer plan 결과의 단계 간 재사용, HTTP 서빙·다중 GPU 환경은 검증하지 않았다. 남은 기간의 보완 계획은 표 14 와 같다.

표 14. 남은 기간의 보완 계획

항목	후속 작업	시기
운영 엔진	vLLM 디코드 단계의 프로파일링으로 attention 시간을 분리하고 가변 길이 손실의 원인 판정	10 월
시뮬레이터	Accel-Sim L2 파티션 해시 변경 후 재검증	10 월
출력 검증	64K 자연어 입력 시나리오로 단일 단계 argmax 변화가 무작위 문맥 특유인지 확인	11 월
선택 정책	FlashInfer plan 결과를 단계 간 재사용할 수 있는지 확인(가능하면 라이브러리 교체 테이블의 손익이 바뀜)	11 월
최종 정리	대시보드 시연 보강, 최종 보고서·시연 준비	12 월

■ 5. 결론 및 소감

가) 결론

본 작품은 가변 길이 LLM 배치에서 FlashAttention-2 와 FlashInfer 의 기본 설정이 모두 요청 길이 분포를 보지 않아 소수의 CTA 에 작업을 집중시키고 GPU 를 충분히 활용하지 못하는 문제를 확인하였다. 공개 트레이스 재생으로 이 문제가 배치 크기 26 이상과 긴 프롬프트가 함께 있을 때 디코드 단계의 13~14%에서 나타나는 조건부 문제임을 보였고, 긴 요청이 64K 이면 커널 손실이 최악 5.18 배로 커짐을 측정하였다. RTX 4090 의 7,443 개 조건 측정과 CTA 작업량·연산 클래스 분해를 바탕으로 선택 정책을 구현했으며, Qwen3-4B 의 가변 길이 조건에서 TPOT 을 61.06 ms 에서 33.80 ms(32K, 생성 토큰 동일), 96.93 ms 에서 40.06 ms(64K, 분기 1 건)로 줄였다.

같은 엔진에 FlashInfer 를 통합한 비교에서 기존 커널의 분할 수만 외부에서 선택해도 커널 교체 이득의 대부분을 얻었고(1.81 배 대 1.83 배), 라이브러리까지 함께 고르는 추가 이득은 약 1%로 단계당 plan 비용이 그 한계였다. 출력 불일치는 분기 지점 수와 ULP 여유로 분류하고 2 ULP 를 넘는 사건은 커널 수준 수치 검사로 판정하여, 검사한 두 사건(네 차례)에서 커널 간 차이가 반올림 수준임을 확인하였다. 개선이 없는 균일 배치, 선택 비용이 이득을 가리는 조건, vLLM 에서 기전이 확인되지 않은 점도 함께 보고하였다. 앞으로 vLLM 단계 프로파일링, 시뮬레이터 보정, 최종 시연을 통해 적용 범위를 확인할 계획이다.

나) 소감

가장 크게 배운 점은 문제를 정확히 정의하는 일이었다. 처음에는 Transformer 와 Mamba 를 상황에 따라 바꾸려 했지만 두 모델은 서로 다른 답을 낸다. 같은 입력의 출력을 유지하면서 실행을 개선한다는 기준으로 돌아가 전환 대상을 커널 설정으로 좁혔고, 이 기준이 이후 실험의 방향을 정했다. 중간보고서 이후에는 "문제가 너무 좁고 이득이 작지 않은가"라는 질문에 답하기 위해 공개 트레이스 재생과 두 라이브러리의 기본 설정 비교, 64K 측정을 추가했고, 그 결과 문제 정의를 "FlashAttention 의 휴리스틱"에서 "길이 분포를 보지 않는 라이브러리 기본 설정"으로 넓히되 발생 조건을 명시하는 쪽으로 고쳤다.

측정 과정에서는 다른 작업이 실행 중이던 공용 GPU 에서 측정한 기록에서 커널 시간이 전체 지연보다 세 배 긴 값을 발견하였다. 해당 결과를 폐기하고 재측정한 뒤 GPU 사용률과 다른 프로세스 기록을 남기도록 바꾸었다. 또한 무인 측정 대기열이 토큰 비동일을 알리는 정상 종료 코드를 실패로 오인하여 결과 폴더를 삭제한 일이 있었다. 이후 실패 판정을 종료 코드가 아니라 결과 기록의 상태로 하고 어떤 결과도 삭제하지 않도록 규약을 바꾸었다. 도구가 출력한 숫자라도 실행 환경과 일관성을 확인해야 하며, 자동화의 판정 기준 자체를 검토해야 한다는 점을 배웠다.

균일 길이에서 이득이 없거나 하이브리드 정책이 첫 결정 비용 때문에 테이블보다 느리게 나온 결과는 처음에는 아쉬웠지만, 적용 조건을 구분하고 선택 비용을 줄이는 근거가 되었다. 출력 불일치를 토큰 일치율로만 보던 것을 분기 지점과 ULP 여유, 커널 수준 검사로 바꾸면서 "답이 바뀌지 않는다"는 주장보다 "어디서 왜 갈리는지"를 보고하는 편이 더 설득력 있다는 점도 배웠다. 다른 사람이 같은 기록에서 수치를 확인할 수 있게 만드는 일도 연구의 일부임을 배웠다.

■ 참고문헌

[1] A. Vaswani et al., "Attention is all you need," NeurIPS, vol. 30, 2017.

[2] G.-I. Yu et al., "Orca: A distributed serving system for Transformer-based generative models," OSDI, pp. 521–538, 2022.

[3] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," SOSP, pp. 611–626, 2023.

[4] T. Dao et al., "Flash-Decoding for long-context inference," PyTorch Blog, 2023. https://pytorch.org/blog/flash-decoding/

[5] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," ICLR, 2024.

[6] A. Bakhoda et al., "Analyzing CUDA workloads using a detailed GPU simulator," ISPASS, pp. 163–174, 2009.

[7] M. Khairy et al., "Accel-Sim: An extensible simulation framework for validated GPU modeling," ISCA, pp. 473–486, 2020.

[8] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[9] A. Agrawal et al., "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," OSDI, 2024.

[10] 김수진·유헌창, 「LLM 추론 성능 최적화를 위한 클러스터링 기반 KV 캐시 재사용 시스템」, 한국정보처리학회 추계학술발표대회, pp. 592–595, 2025.

[11] 최우형 외, 「거대 언어 모델을 서빙하기 위한 GPU 메모리 계층화 사례 연구」, 정보과학회지, 43(4), pp. 37–40, 2025.

[12] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," NeurIPS, vol. 35, 2022.

[13] K. Hong et al., "FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics," MLSys, 2024.

[14] Z. Ye et al., "FlashInfer: Efficient and customizable attention engine for LLM inference serving," MLSys, 2025.

[15] S. Williams et al., "Roofline: An insightful visual performance model for multicore architectures," Commun. ACM, 52(4), pp. 65–76, 2009.

[16] 정종현 외, 「최신 GPU 아키텍처의 데이터 캐시 성능 분석」, 한국소프트웨어종합학술대회, pp. 1052–1054, 2021.

[17] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," COLM, 2024.

[18] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," ISCA, 2024. (Azure LLM Inference Trace 2023, github.com/Azure/AzurePublicDataset)

[19] Y. Wang et al., "BurstGPT: A real-world workload dataset to optimize LLM serving systems," arXiv:2401.17644, 2024.

[20] R. Sanovar et al., "Lean Attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers," MLSys, 2025.

[21] A. Agrawal et al., "Vidur: A large-scale simulation framework for LLM inference," MLSys, 2024.

[22] vLLM Project, "vllm.v1.attention.backends.flash_attn," vLLM v0.9.2 API documentation, 2025. https://docs.vllm.ai/en/v0.9.2/api/vllm/v1/attention/backends/flash_attn.html

■ 부록 A. 원천코드 주요 부분 발췌

전체 코드: https://github.com/sunjae0224/kernelscope

A.1 분할 수 휴리스틱 kernelscope/model/geometry.py

```
def num_splits_heuristic(batch_nheads_mblocks, num_sms, num_n_blocks, max_splits=128):
    """Exact port of flash-attn 2.8.3 num_splits_heuristic (flash_api.cpp)."""
    if batch_nheads_mblocks >= 0.8 * num_sms:
        return 1
    max_splits = min(max_splits, num_sms, num_n_blocks)
    eff = [0.0] * (max_splits + 1)
    best = 0.0
    for s in range(1, max_splits + 1):
        if s > 1 and math.ceil(num_n_blocks / s) == math.ceil(num_n_blocks / (s - 1)):
            continue
        n_waves = batch_nheads_mblocks * s / num_sms
        eff[s] = n_waves / math.ceil(n_waves)
        best = max(best, eff[s])
    for s in range(1, max_splits + 1):
        if ((s == 1 or math.ceil(num_n_blocks / s) !=
                math.ceil(num_n_blocks / (s - 1)))
                and eff[s] >= 0.85 * best):
            return s
    return 1
```

A.2 조회 테이블 선택 kernelscope/serve/dispatch.py

```
class TablePolicy:
    @staticmethod
    def _features(lens):
        return np.log2([len(lens), max(lens), float(np.mean(lens))])

    def choose(self, lens, n_heads, n_kv_heads) -> int:
        _validate(lens, n_heads, n_kv_heads)
        key = (n_heads, n_kv_heads, tuple(math.ceil(n / PAGE) for n in lens))
        self.last_cache_hit = key in self._cache
        if self.last_cache_hit:
            value, self.last = self._cache[key]
            return value
        eligible = self._groups.get((n_heads, n_kv_heads), [])
        if not eligible:
            raise ValueError(
                f"dispatch table has no measured shape with Hq={n_heads}, "
                f"Hkv={n_kv_heads}")
        query = self._features(lens)
        distances = np.square(
            self._features_by_shape[(n_heads, n_kv_heads)] - query
        ).sum(axis=1)
        index = int(np.argmin(distances))
        workload, value = eligible[index]
        self.last = {"workload_key": workload.key(),
                     "distance": float(distances[index] ** 0.5)}
        self._cache[key] = (value, self.last)
        return value
```

A.3 하이브리드 선택 kernelscope/serve/dispatch.py

```
class HybridPolicy:
    """Model ranking, but the measured table's variant wins when the model predicts it within
``delta``."""
    def choose(self, lens, n_heads, n_kv_heads) -> int:
        model_splits = self.model.choose(lens, n_heads, n_kv_heads)
        predicted = dict(self.model.last)
        if not self.table.covers(n_heads, n_kv_heads):   # no measured grid for this head shape
            self.last = {"source": "model", "pick": min(predicted, key=predicted.get)}
            return model_splits
        table_splits = self.table.choose(lens, n_heads, n_kv_heads)
        table_variant = variant_for_splits(table_splits)
        pick = hybrid_pick(predicted, table_variant, self.delta)
        source = "table" if pick == table_variant else "model"
        self.last = {"source": source, "pick": pick,
                     "model": predicted, "table": self.table.last}
        return table_splits if source == "table" else model_splits
```

A.4 FlashInfer 디코드 백엔드 kernelscope/serve/attention.py

```
class FlashInferDecode:
    def plan(self, cache_lens, block_table, n_heads, n_kv_heads, head_dim, dtype) -> float:
        """Plan this step's kernel for the lengths after the append."""
        started = time.perf_counter()
        lens = [int(n) + 1 for n in cache_lens.tolist()]
        indptr, indices, last = csr_from_block_table(block_table, lens, self.page)
        ...
        self.wrapper.plan(indptr.to(device), indices.to(device), last.to(device),
                          n_heads, n_kv_heads, head_dim, self.page, q_data_type=dtype,
                          kv_data_type=dtype, sm_scale=head_dim ** -0.5)
        torch.cuda.synchronize(device)
        return (time.perf_counter() - started) * 1e6

    def __call__(self, q, k_cache, v_cache, *, k, v, cache_seqlens, block_table,
                 softmax_scale, causal, num_splits):
        if q.shape[1] != 1:              # prefill stays on flash_attn_with_kvcache
            return self.fallback(q, k_cache, v_cache, k=k, v=v, cache_seqlens=cache_seqlens,
                                 block_table=block_table, softmax_scale=softmax_scale,
                                 causal=causal, num_splits=num_splits)
        k_cache[self._pages, self._slots] = k[:, 0]
        v_cache[self._pages, self._slots] = v[:, 0]
        return self.wrapper.run(q[:, 0], (k_cache, v_cache)).unsqueeze(1)
```